

1. Develop a lexical Analyzer to identify identifiers, constants, operators using C program

```
#include<stdio.h>

#include<ctype.h>

#include<string.h>

int main()

{

int i,ic=0,m,cc=0,oc=0,j;

char b[30],operators[30],identifiers[30],constants[30];

printf("enter the string : ");

scanf("%[^\n]s",&b);

for(i=0;i<strlen(b);i++)

{

if(isspace(b[i]))

{

continue;

}

else if(isalpha(b[i]))

{

identifiers[ic] =b[i];

ic++;

}

else if(isdigit(b[i]))

{

m=(b[i]-'0');

i=i+1;

while(isdigit(b[i]))

{
```

```
m=m*10 + (b[i]-'0');  
i++;  
}  
i=i-1;  
constants[cc]=m;  
cc++;  
}  
else  
{  
    if(b[i]=='*')  
    {  
        operators[oc]='*';  
        oc++;  
    }  
    else if(b[i]=='-')  
    {  
        operators[oc]='-';  
        oc++;  
    }  
    else if(b[i]=='+')  
    {  
        operators[oc]='+';  
        oc++;  
    }  
    else if(b[i]=='=')  
    {  
        operators[oc]='=';  
        oc++;  
    }
```

```

    }
}
}

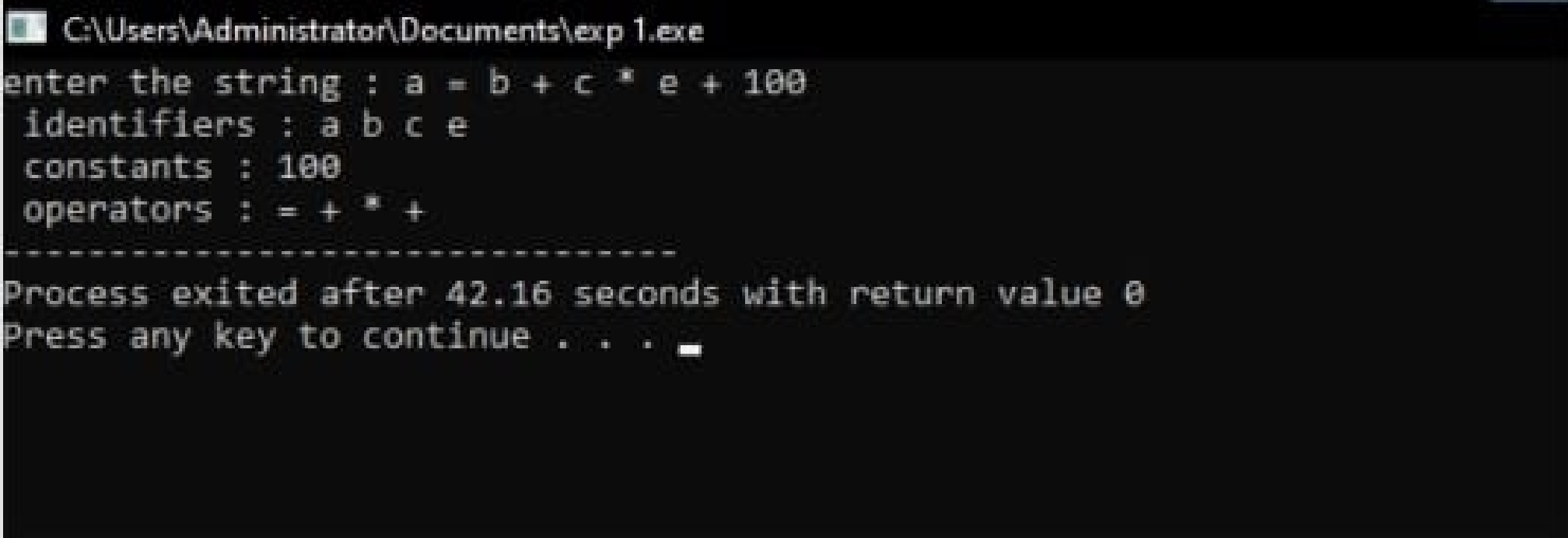
printf(" identifiers : ");
for(j=0;j<lc;j++)
{
    printf("%c ",identifiers[j]);
}

printf("\n constants : ");
for(j=0;j<cc;j++)
{
    printf("%d ",constants[j]);
}

printf("\n operators : ");
for(j=0;j<oc;j++)
{
    printf("%c ",operators[j]);
}
}
}

```

OUTPUT:



```

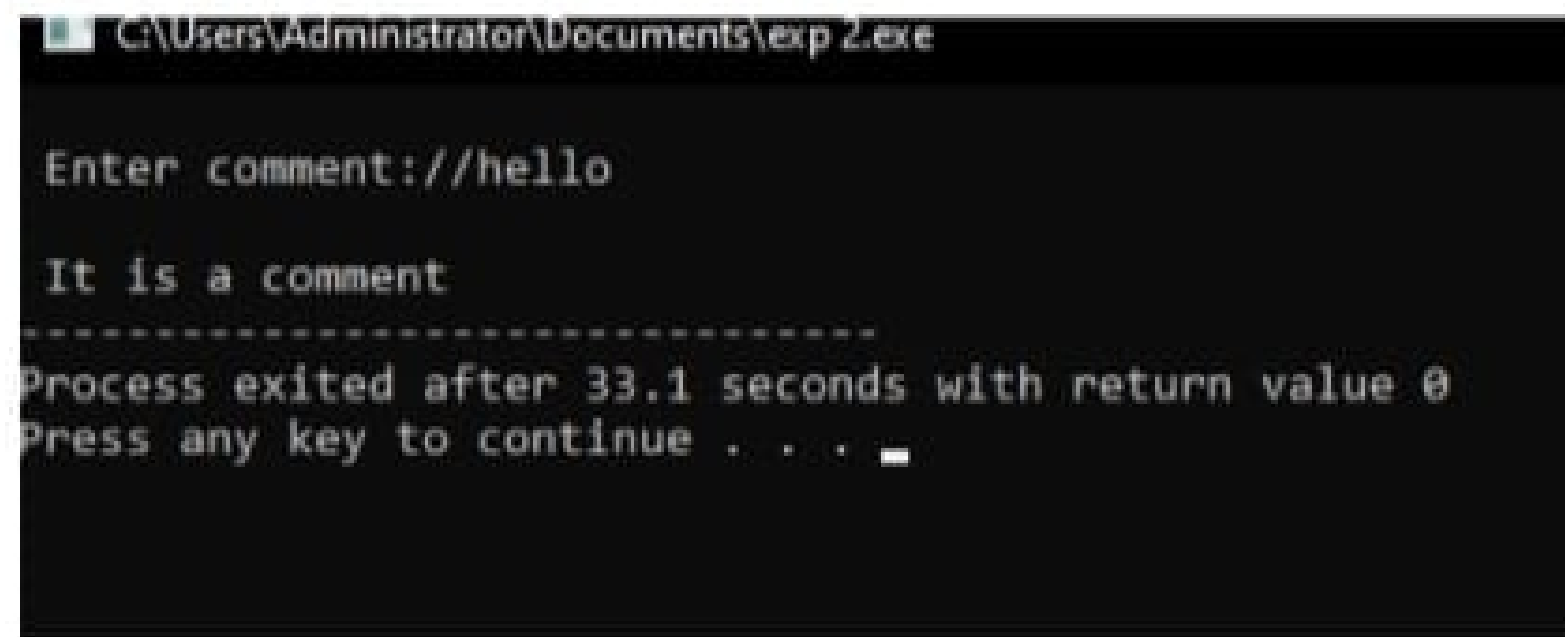
C:\Users\Administrator\Documents\exp 1.exe
enter the string : a = b + c * e + 100
identifiers : a b c e
constants : 100
operators : = + * +
-----
Process exited after 42.16 seconds with return value 0
Press any key to continue . . . 

```

2. Develop a lexical Analyzer to identify whether a given line is a comment or not using C

```
#include<stdio.h>
#include<conio.h>
int main()
{
    char com[30];
    int i=2,a=0;
    printf("\n Enter comment:");
    gets(com);
    if(com[0]=='/')
    {
        if(com[1]=='/')
            printf("\n It is a comment");
        else if(com[1]=='*')
        {
            for(i=2;i<=30;i++)
            {
                if(com[i]=='*'&&com[i+1]=='/')
                {
                    printf("\n It is a comment");
                    a=1;
                    break;
                }
            }
            else
                continue;
        }
        if(a==0)
            printf("\n It is not a comment");
    }
    else
        printf("\n It is not a comment");
    }
    else
        printf("\n It is not a comment");
    }
```

OUTPUT:



```
C:\Users\Administrator\Documents\exp 2.exe

Enter comment: //hello

It is a comment
-----
Process exited after 33.1 seconds with return value 0
Press any key to continue . . .
```

3. Design a lexical Analyzer for given language should ignore the redundant spaces, tabs and new lines and ignore comments using C

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

int isKeyword(char buffer[]){
char keywords[32][10] =
{"main","auto","break","case","char","const","continue","default",
"do","double","else","enum","extern","float","for","goto",
"if","int","long","register","return","short","signed",
"sizeof","static","struct","switch","typedef",
"unsigned","void","printf","while"};

int i, flag = 0;
for(i = 0; i < 32; ++i)
{
if(strcmp(keywords[i], buffer) == 0)
{
flag = 1;
break;
}
```

```

    }
}
return flag;
}
int main()
{
    char ch, buffer[15], operators[] = "+-*/%=";
    FILE *fp;
    int i,j=0;
    fp = fopen("flex_input.txt","r");
    if(fp == NULL){
        printf("error while opening the file\n");
        exit(0);
    }
    while((ch = fgetc(fp)) != EOF){
        for(i = 0; i < 6; ++i){
            if(ch == operators[i])
                printf("%c is operator\n", ch);
        }

        if(isalnum(ch)){
            buffer[j++] = ch;
        }
        else if((ch == ' ' || ch == '\n') && (j != 0)){
            buffer[j] = '\0';
            j = 0;

            if(isKeyword(buffer) == 1)

```

```

if (str[i] == ' ' || str[i] == '\t') {

words++;

} else if (str[i] == '\n') {

lines++;

} else {

characters++;

}}

// Check for an empty input

if (characters > 0) {

words++; // If there are characters, there is at least one word

lines++; // If there are characters, there is at least one line

}

printf("Total number of words: %d\n", words);

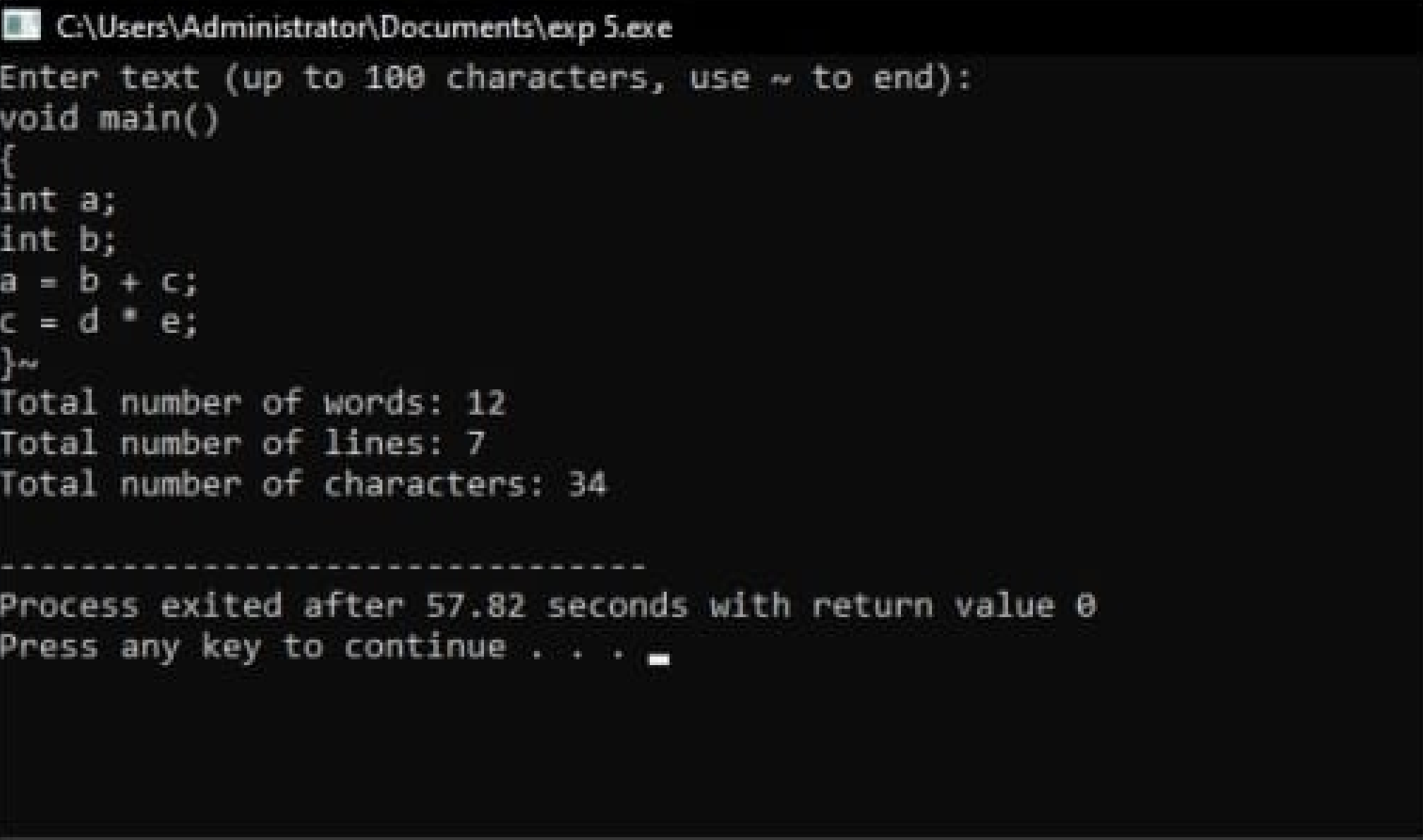
printf("Total number of lines: %d\n", lines);

printf("Total number of characters: %d\n", characters);

return 0;

```

}OUTPUT



```

C:\Users\Administrator\Documents\exp 5.exe
Enter text (up to 100 characters, use ~ to end):
void main()
{
int a;
int b;
a = b + c;
c = d * e;
}~
Total number of words: 12
Total number of lines: 7
Total number of characters: 34

-----
Process exited after 57.82 seconds with return value 0
Press any key to continue . . .

```

4. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular arithmetic operators using C

```
#include<stdio.h>

#include<conio.h>

int main()

{

char s[5];

printf("\n Enter any operator:");

gets(s);

switch(s[0])

{

case '>':

if(s[1]=='=')

printf("\n Greater than or equal");

else

printf("\n Greater than");

break;

case '<':

if(s[1]=='=')

printf("\n Less than or equal");

else

printf("\n Less than");

break;

case '=':

if(s[1]=='=')

printf("\n Equal to");

else

printf("\n Assignment");
```

```
break;

case '!':

if(s[1]=='=')

printf("\nNot Equal");

else

printf("\n Bit Not");

break;

case '&':

if(s[1]=='&')

printf("\nLogical AND");

else

printf("\n Bitwise AND");

break;

case '|':

if(s[1]=='|')

printf("\nLogical OR");

else

printf("\nBitwise OR");

break;

case '+':

printf("\n Addition");

break;

case '-':

printf("\nSubstraction");

break;

case '*':

printf("\nMultiplication");

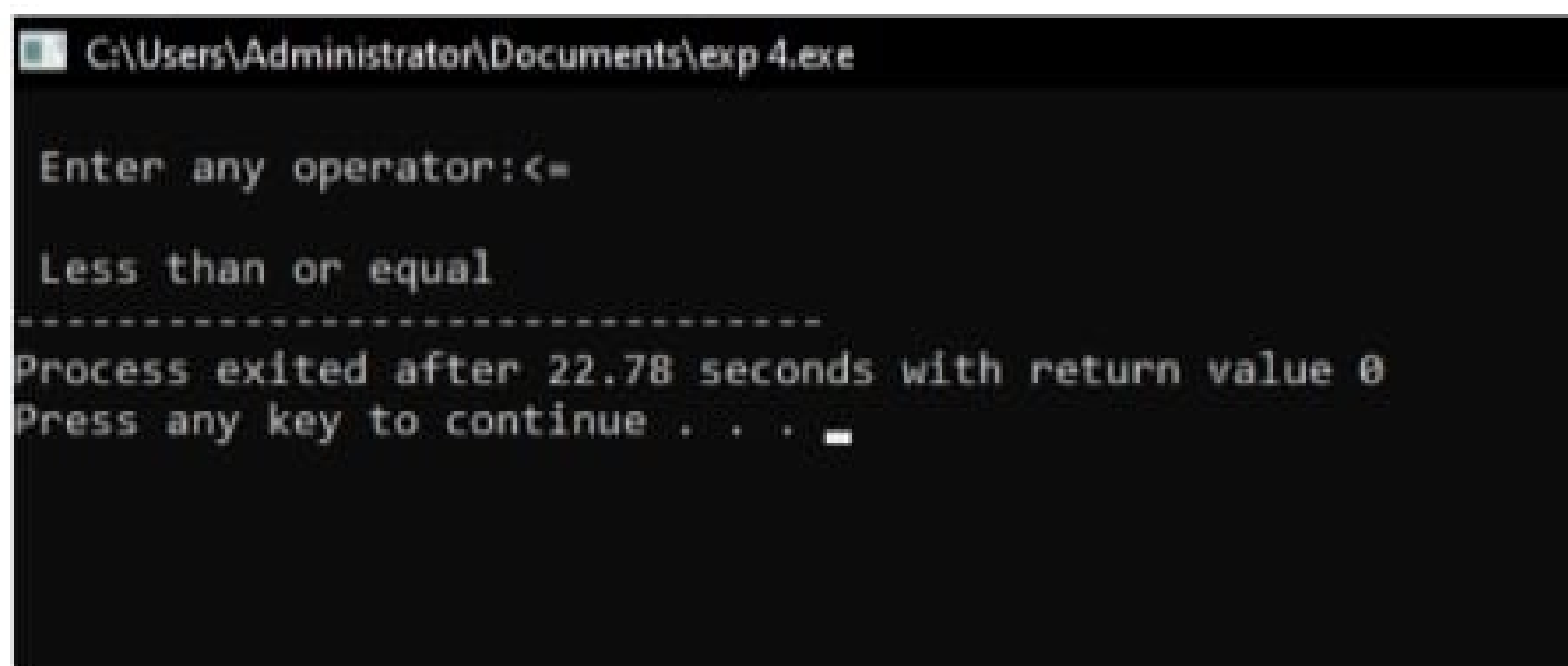
break;
```

```

case '/':
printf("\nDivision");
break;
case '%':
printf("Modulus");
break;
default:
printf("\n Not a operator");
}
}

```

OUTPUT:



```

C:\Users\Administrator\Documents\exp 4.exe

Enter any operator:<=
Less than or equal
-----
Process exited after 22.78 seconds with return value 0
Press any key to continue . . .

```

5. Design a lexical Analyzer to find the number of whitespaces and newline characters using C.

```

#include <stdio.h>

int main() {

char str[100];

int words = 0, lines = 0, characters = 0;

printf("Enter text (up to 100 characters, use ~ to end):\n");

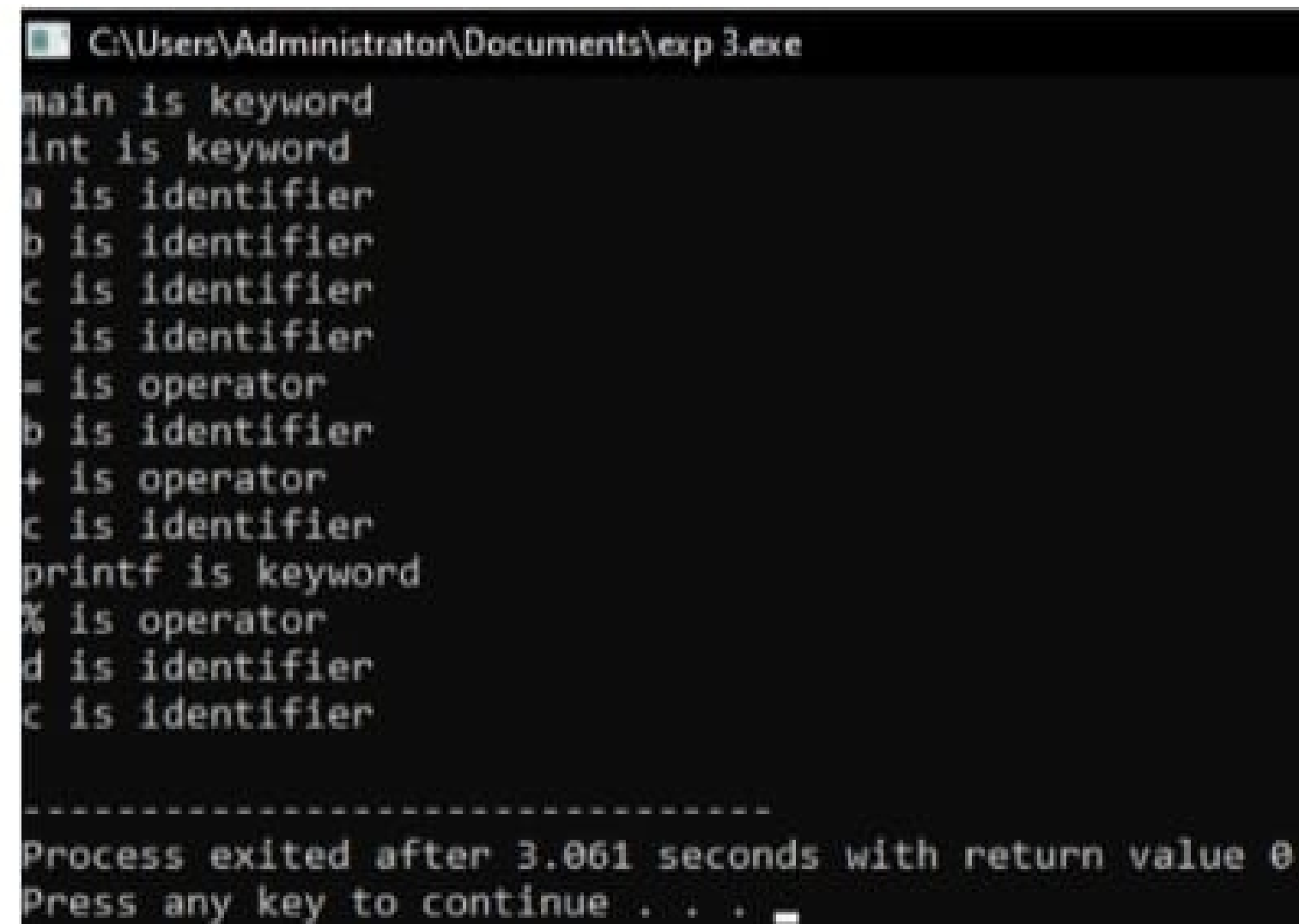
scanf("%[^\n]", str);

for (int i = 0; str[i] != '\0'; i++) {

```

```
printf("%s is keyword\n", buffer);  
else  
printf("%s is identifier\n", buffer);  
}  
  
}  
fclose(fp);  
return 0;  
}
```

OUTPUT:



```
C:\Users\Administrator\Documents\exp 3.exe  
main is keyword  
int is keyword  
a is identifier  
b is identifier  
c is identifier  
c is identifier  
= is operator  
b is identifier  
+ is operator  
c is identifier  
printf is keyword  
% is operator  
d is identifier  
c is identifier  
  
-----  
Process exited after 3.061 seconds with return value 0  
Press any key to continue . . .
```